Mid-term Project Requirements, Questions and Answers

Mid-term Project Definition

The project you are designing here is the **Mid-Term Project Specifications** — the grants.gov solicitation harvester and VT College of Science team builder. Read it end to end (requirements, use cases, deliverables, rubric, and the *Questions for Consideration*) before you start this exercise.

Assignment Overview

Anyone can build an agentic system with Claude Code or Open Code. *Can you build one that is reliable, safe, cost effective and meets all the requirements every time it's run?* It's time to put on your AI architect hat and address LLM/RAG shortcomings with a good agentic AI design.

Rubric

This is a 500 point exercise. It's an assessment of how much you have learned up to this point. Please take it seriously. It's as important as the project output. You should complete this Quarto markdown file (.qmd) and turn it in along (as .qmd) with a quarto rendered PDF version (.pdf). Add `format: pdf` to the yaml header of this file and then run `quarto render yourfilename.qmd` to build the PDF.

Important Note

This work must be done 100% without assistance from any AI. I need to know that *YOU* have learned the material. I know that Claude knows it. **Any use of AI will be easily detected and will be treated as an Honor Code violation. Please take this seriously.**

Requirements

Refer back to the course material in the `Projects` section and complete this list of requirements. List as many as you can (20 is not meaningful here). This is a critical step! Any person or LLM should be able to fully understand **all** requirements for the application from this list.

Complete this list:

1. The solution should provide a user-friendly user interface in Python streamlit.
2. The solution should harvest **open** solicitations from grants.gov via the REST API and store them in a local JSON database.
3. The solution should take the **closed** solicitations from the grants.gov REST API and archive them in said local JSON database.
4. The solution should allow the user to filter and sort the **open** solicitations by relevant fields (type, date, amount, etc.).
5. The solution should truncate the **open** solicitations by a set number displayed per page as to not have one long list, along with a description of how many **open** solicitations and time of last update.
6. The solution should allow the user to click a button that builds the team **after** selecting a solicitation.
7. The solution should allow the user to refresh the list of solicitations, calling the grants.gov REST API and properly archiving the closed solicitations and displaying the open ones, all while storing them in the local JSON database.
8. The solution should implement a Retrieval-Augmented Generation (RAG) architecture that retrieves chunked data from **raw_data/authors.csv**. These data contains a list of College of Science faculty with their respective descriptions and important metrics, such as the H-index.
9. The solution should also utilize **raw_data/publications.csv** in its RAG, which contains all the research publications for each College of Science faculty member, including abstracts. The initial prototype will utilize **raw_data/publications.csv**, but the final product should utilize the API sourced from Elsevier.

10. The solution should then transfer the user to a chatbot interface that automatically constructs the team based on the optimized user-defined prompt template and RAG content.
11. The solution should use the hosted VT ARC LLMs.
12. The solution should allow the user to follow up on the initial generation of the team.
13. The solution should take any follow-ups, such as swapping team members and scope and retrieve a revised team.
14. The solution should provide well-structured code with descriptive comments and run instructions.
15. The solution should take a holistic approach to building the team. Contributor metrics, along with publications (including recency and journal prestige) at a minimum should be inputs into the team construction and role definitions.
16. The solution should optimize the original prompt using LLM prompt optimization (to help achieve the other requirements in this list).
17. The solution should verify all facts presented through direct grounding in the source materials. The user should be able to trace back any factual claims to the original source material text.
18. The solution should highly reliable. Running the same prompt over again should generate relatively consistent results. If the results are wildly different, the solution is a complete failure and will never be accepted.

LLM / RAG Issues

Referring back to the lectures on LLM and RAG, what are the specific issues that you must overcome with good agentic AI design/architecture/patterns **to meet the requirements**? **Make sure you point back to a specific requirement**. Don't solve the issue yet. Just identify it. You should have a fair number of these (between 5 and 20).

1. For requirements 8 and 9, using a suboptimal chunking strategy of these data can provide questionable results due to chunks containing the correct keyword or the answer splitting across two chunks.
2. For requirement 10, documents retrieved in context that are in the middle often times can be overlooked.
3. For requirement 13, any additional follow ups that are contradictory to the initial prompt can confuse the LLM/RAG on what should be retrieved. Additionally, more prompts means more context used, which can lead to more hallucinated results.
4. For requirement 17, the issue that needs to be addressed is verification. More specifically, what's the best framework we can use to minimize hallucinations?
5. For requirement 18, RAG can produce different outputs even with the same prompt, k_in, k_out, what's the best way to work around this?

Basic Architecture

Write up a couple of paragraphs (in prose) on how you plan to design the solution. What will be the general approach. Don't solve the issues yet… just general approach. What are the major pieces, how do they connect, etc.?

The solution will be architected as a RAG system using ChromaDB to store the embedded chunks from **authors.csv** and **publications.csv** with an LLM re-ranker along with Python's streamlit for the user interface. The initial prototype will utilize **publications.csv** as opposed to an API connected to Elsevier's endpoint, but the final product will use Elsevier. With **authors.csv** and **publications.csv**, an embedding model, qwen3-embedding:8b, hosted on the ADS server, will be used to embed the raw data on a row-by-row basis to prevent data leakage from one professor's data to another. The chunks will be stored in a ChromaDB vector store along with its respective metadata. This is a one time process unless new professor data comes in, in which, the pipeline can be ran again. For retrieval, the query will be embedded with the same model used to embed the authors and publications data and use an ANN search which will then return a top-k number of chunks. A re-ranker will then be used to rank the returned chunks.

For retrieving data from grants.gov, an API will be used to retrieve listings and their respective descriptions that are both open and closed and store into local JSON database, but only display the open opportunities. Now, the streamlit UI will be built with all the necessary filter and display options, along with the 'Build' button after selecting a single opportunity. Implement the LLM to optimize the user-prompt and initiate the RAG process. After the output is given, allow the user to give clarifications or drop a specific team member.

25 Bonus points for including a UML sequence diagram using a mermaid diagram. For this syntax (and only this syntax), I will allow LLM usage** but you need to prompt the LLM with the sequence diagram flow, actors and steps. Include your prompt here.**

Prompt:

Create a UML sequence diagram in Mermaid for a RAG system that builds a team of professors based off a selected grant that shows the diagram flow, actors, and steps. Separate the workflows into three diagrams: research-data ingestion, grant data ingestion, team generation + refinement.

Actors

Include: User, Streamlit UI, Local JSON database, prompt optimization LLM, LLM reranker, ChromaDB, Grants.gov API, Elsevier API, embedding model (qwen3-embedding:8b), team-generation LLM, authors.csv and publications.csv source.

Research data ingestion

1. Retrieve data from authors.csv and publications.csv for the prototype. Use an alt block to show use of Elsevier API for final prototype instead of publications.csv.
2. Send data to embedding model to chunk.
3. Store results into ChromaDB

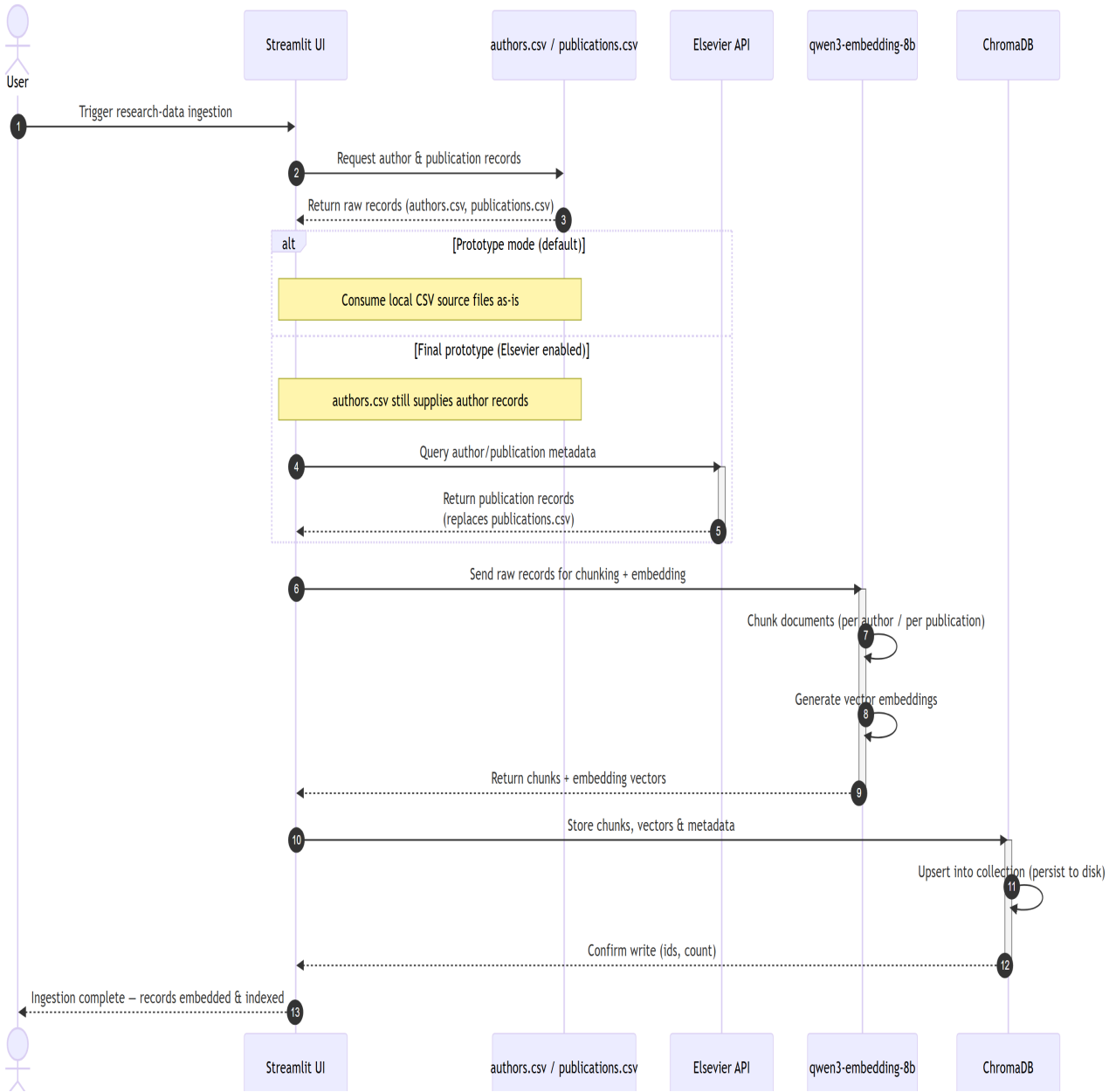
Grant data ingestion

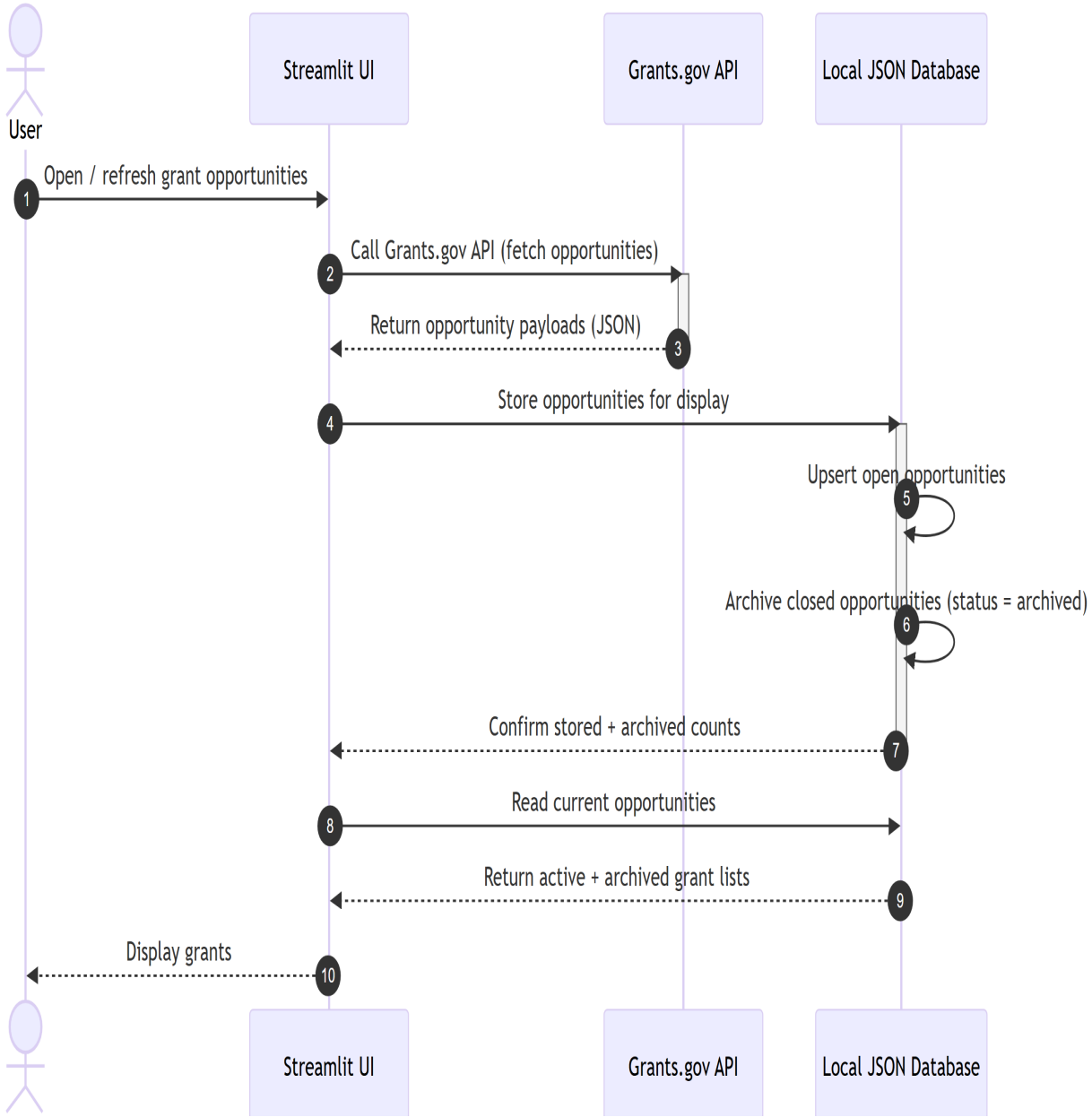
1. Call Grants.gov API for grant data.
2. Store opportunities into local JSON database for display, archive closed ones.

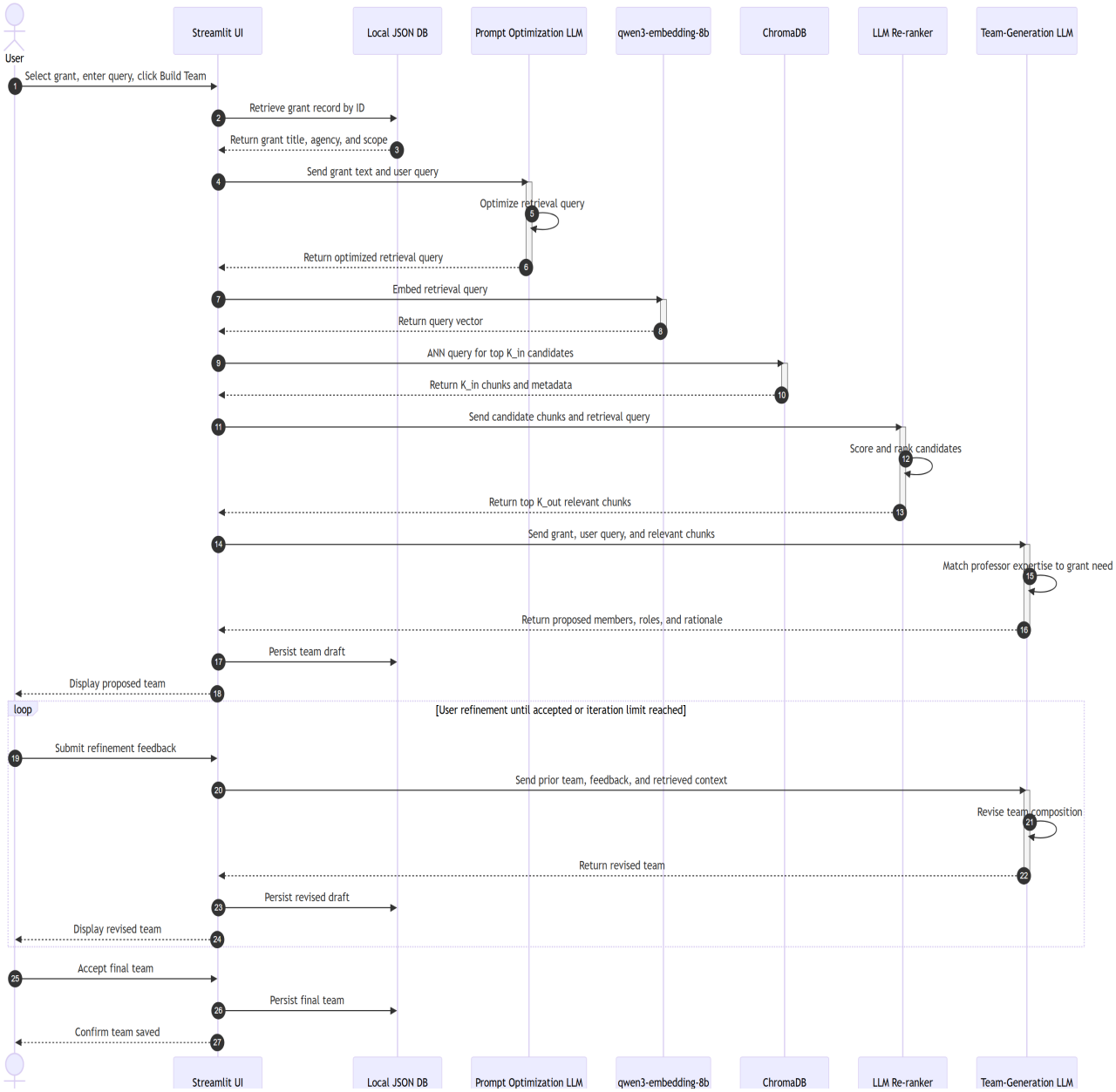
Team generation and refinement

1. The user clicks the build button to initiate call
2. Streamlit retrieves grant from local JSON DB.
3. The prompt optimizer takes grant and user query to create the optimal response.
4. Embed the retrieval query through the same embedding model as before
5. Query ChromaDB through ANN to retrieve K_in
6. Send queried chunks to reranker.
7. The Team Generation LLM takes relevant chunks and proposes a team.
8. Send response back to user.
9. Include for loop for user refinement.
10. Display revised team

vt-arc-llm


User
Streamlit UI
authors.csv / publications.csv
Elsevier API
qwen3-embedding-8b
ChromaDB
Trigger research-data ingestion
Request author & publication records
Return raw records (authors.csv, publications.csv)
alt
[Prototype mode (default)]
Consume local CSV source files as-is
[Final prototype (Elsevier enabled)]
authors.csv still supplies author records
Query author/publication metadata
Return publication records
(replaces publications.csv)
Send raw records for chunking + embedding
Chunk documents (per author / per publication)
Generate vector embeddings
Return chunks + embedding vectors
Store chunks, vectors & metadata
Upsert into collection (persist to disk)
Confirm write (ids, count)
Ingestion complete – records embedded & indexed


User
Streamlit UI
Grants.gov API
Local JSON Database
1
Open / refresh grant opportunities
2
Call Grants.gov API (fetch opportunities)
3
Return opportunity payloads (JSON)
4
Store opportunities for display
5
Upsert open opportunities
6
Archive closed opportunities (status = archived)
7
Confirm stored + archived counts
8
Read current opportunities
9
Return active + archived grant lists
10
Display grants
Streamlit UI
Grants.gov API
Local JSON Database


User
Streamlit UI
Local JSON DB
Prompt Optimization LLM
qwen3-embedding-8b
ChromaDB
LLM Re-ranker
Team-Generation LLM
1 Select grant, enter query, click Build Team
2 Retrieve grant record by ID
3 Return grant title, agency, and scope
4 Send grant text and user query
5 Optimize retrieval query
6 Return optimized retrieval query
7 Embed retrieval query
8 Return query vector
9 ANN query for top K_in candidates
10 Return K_in chunks and metadata
11 Send candidate chunks and retrieval query
12 Score and rank candidates
13 Return top K_out relevant chunks
14 Send grant, user query, and relevant chunks
15 Match professor expertise to grant need
16 Return proposed members, roles, and rationale
17 Persist team draft
18 Display proposed team
loop
[User refinement until accepted or iteration limit reached]
19 Submit refinement feedback
20 Send prior team, feedback, and retrieved context
21 Revise team composition
22 Return revised team
23 Persist revised draft
24 Display revised team
25 Accept final team
26 Persist final team
27 Confirm team saved

You should end your `Basic Architecture` with the level of agency you think is **earned** by this solution architecture. Go back to the beginning of the course for the various levels. **Make sure you give a quick summary of why it is earned.**

The level of agency earned by this solution architecture is a **deterministic workflow**. This is due to the predefined steps taken when executing the research-data ingestion and team building. For example, the team building follows a deterministic path of prompt ingestion/optimization, retrieval of necessary chunks from ChromaDB, re-ranking, and providing user with the proposed team with its rationale. The solution allows the user to follow-up, but overall the system follows a step-by-step process to come up with a solution. This architecture gives the developer control over the execution sequence and termination, making step count predictable, along with creating estimates/bounds for cost and token consumption. One drawback of this design is that it might be **too rigid**, i.e., an edge case that the predefined steps doesn't know how to handle.

Solution Mitigation

Now for each of the LLM/RAG issues, you should give specific design patterns that we learned in class to address them. I need a short explanation of why your solution is the right one. **Make sure you refer back to the numbered issue from the previous section.**

1. For issue 1, the design pattern that addresses this issue is utilizing **document-aware** chunking. This strategy preserves the structural integrity of the .csvs by insuring that no row is split across chunks. Also, it would maintain the meaning of the headers. In doing so, it would prevent data about a publication or professor leaking into one another.
2. For issue 2, the design pattern that addresses this issue is using a LLM re-ranker such that the most critical and relevant information comes first or last, and to keep top_k smaller. This is the optimal approach as it allows the LLM to overcome the lost-in-the-middle problem and to cut down on irrelevant information.
3. For issue 3, the design pattern that addresses this issue is using a **rolling summary** of the context. This will keep a summary of what is asked from the user, along with any other conflicting clarifications that may arise. This is the optimal solution as truncating may remove necessary information from previous prompts and leaving it be may cause hallucinations.
4. For issue 4, the design pattern that addresses this issue is to utilize the Chain-of-Verification framework to verify the response. Essentially, give a context-free LLM a list of questions about the original claim and execute the retrieval again. This approach is optimal because it targets where hallucinations live, like a citation. Additionally,

the independence of the original response is not in context, so it makes it more difficult to agree with itself.

5. For issue 5, using many different strategies such as keeping a low temperature, maintaining consistent k_in and k_out, using the same embedding model, re-ranker, and version of the corpus and vector index, would help support consistent answers. This approach is optimal it reduces how much is changed from run-to-run at each stage.

Final Thoughts

Hopefully, you can see that this exercise is something you do before invoking an agentic harness. This is where you add value and earn your place in the future AI ecosystem. Next steps we will review this architecture with a frontier level model and refine it. Then we work with an agentic harness to build it!